

# MANUAL DE PROCESO DE INGENIERÍA





# Fase 4: Programación y Electrónica

## ÍNDICE

### **propósito, estructura y objetivo**

- 1.1 Propósito del área
- 1.2 Roles y responsabilidades
- 1.3 Filosofía

### **Protocolo de electrónica y cableado**

- 2.1 diseño y planificación de la estructura eléctrica
- 2.2 estándares de cableado y conexiones
- 2.3 verificación de conexiones y encendido seguro

### **Protocolo de desarrollo**

- 3.1 plataformas y entornos de desarrollo
- 3.2 control de versiones y colaboración
- 3.3 estándares estructurales

### **protocolo de pruebas, calibración y debugging**

- 4.1 Pirámide de pruebas
- 4.2 Tuning
- 4.3 Pruebas y calibración de autónomo
- 4.4 fine tuning

### **Transición a competencia y soporte en pits**

- 5.1 Preparación del código
- 5.2 Checklist para pits

### **Documentación**

- 6.1 Propósito de la Documentación
- 6.2 Engineering Notebook (Libro de Ingeniería)
- 6.3 Bitácora Técnica del Equipo
- 6.4 Documentación de Autónomos y Rutinas
- 6.5 Documentación de Código y Sistema
- 6.6 Tech Binder (Opcional)
- 6.7 Respaldo y Control de Información





# PROPÓSITO, ESTRUCTURA Y OBJETIVO

## 1.1 Propósito del Área

- Diseñar, desarrollar e integrar el sistema de control del robot (hardware + software) garantizando:
- Control preciso de subsistemas
- Comunicación estable entre dispositivos
- Capacidad de adaptación en competencia

## 1.2 Roles y Responsabilidades

- Líder de Software

Define arquitectura del código

Aprueba Pull Requests

Responsable del deployment final

- Líder de Electrónica

Diseña sistema eléctrico

Supervisa cableado y seguridad

Válida encendidos

- Programadores de Subsistemas

Implementan lógica de cada mecanismo

Realizan pruebas unitarias

- Técnico de Integración

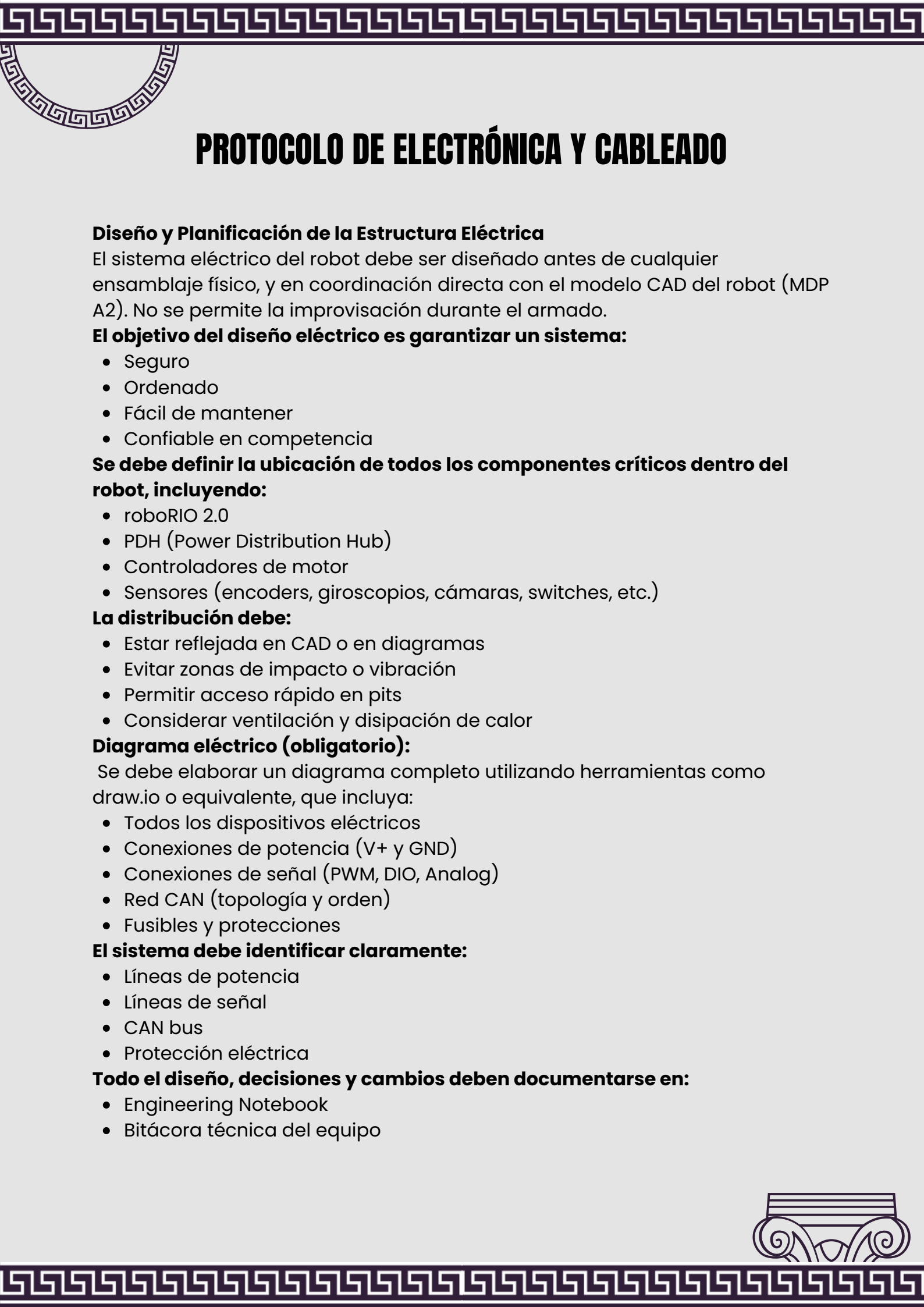
Coordina pruebas hardware/software

Detecta fallas en interacción entre sistemas

## 1.3 Filosofía de Trabajo

- Todo cambio debe ser reproducible
- Todo sistema debe poder explicarse
- Todo componente debe estar documentado
- Nada se conecta sin haber sido planeado





# PROTOCOLO DE ELECTRÓNICA Y CABLEADO

## Diseño y Planificación de la Estructura Eléctrica

El sistema eléctrico del robot debe ser diseñado antes de cualquier ensamblaje físico, y en coordinación directa con el modelo CAD del robot (MDP A2). No se permite la improvisación durante el armado.

**El objetivo del diseño eléctrico es garantizar un sistema:**

- Seguro
- Ordenado
- Fácil de mantener
- Confiable en competencia

**Se debe definir la ubicación de todos los componentes críticos dentro del robot, incluyendo:**

- roboRIO 2.0
- PDH (Power Distribution Hub)
- Controladores de motor
- Sensores (encoders, giroscopios, cámaras, switches, etc.)

**La distribución debe:**

- Estar reflejada en CAD o en diagramas
- Evitar zonas de impacto o vibración
- Permitir acceso rápido en pits
- Considerar ventilación y disipación de calor

**Diagrama eléctrico (obligatorio):**

Se debe elaborar un diagrama completo utilizando herramientas como draw.io o equivalente, que incluya:

- Todos los dispositivos eléctricos
- Conexiones de potencia (V+ y GND)
- Conexiones de señal (PWM, DIO, Analog)
- Red CAN (topología y orden)
- Fusibles y protecciones

**El sistema debe identificar claramente:**

- Líneas de potencia
- Líneas de señal
- CAN bus
- Protección eléctrica

**Todo el diseño, decisiones y cambios deben documentarse en:**

- Engineering Notebook
- Bitácora técnica del equipo





## Estándares de Cableado y Conexiones

El cableado del robot debe seguir estándares estrictos para asegurar confiabilidad, facilidad de mantenimiento y rapidez en diagnóstico.

Código de colores (obligatorio):

- Rojo → Voltaje positivo (V+)
- Negro → Tierra (GND)
- Amarillo / Azul / Verde → Señales

## Buenas prácticas de cableado:

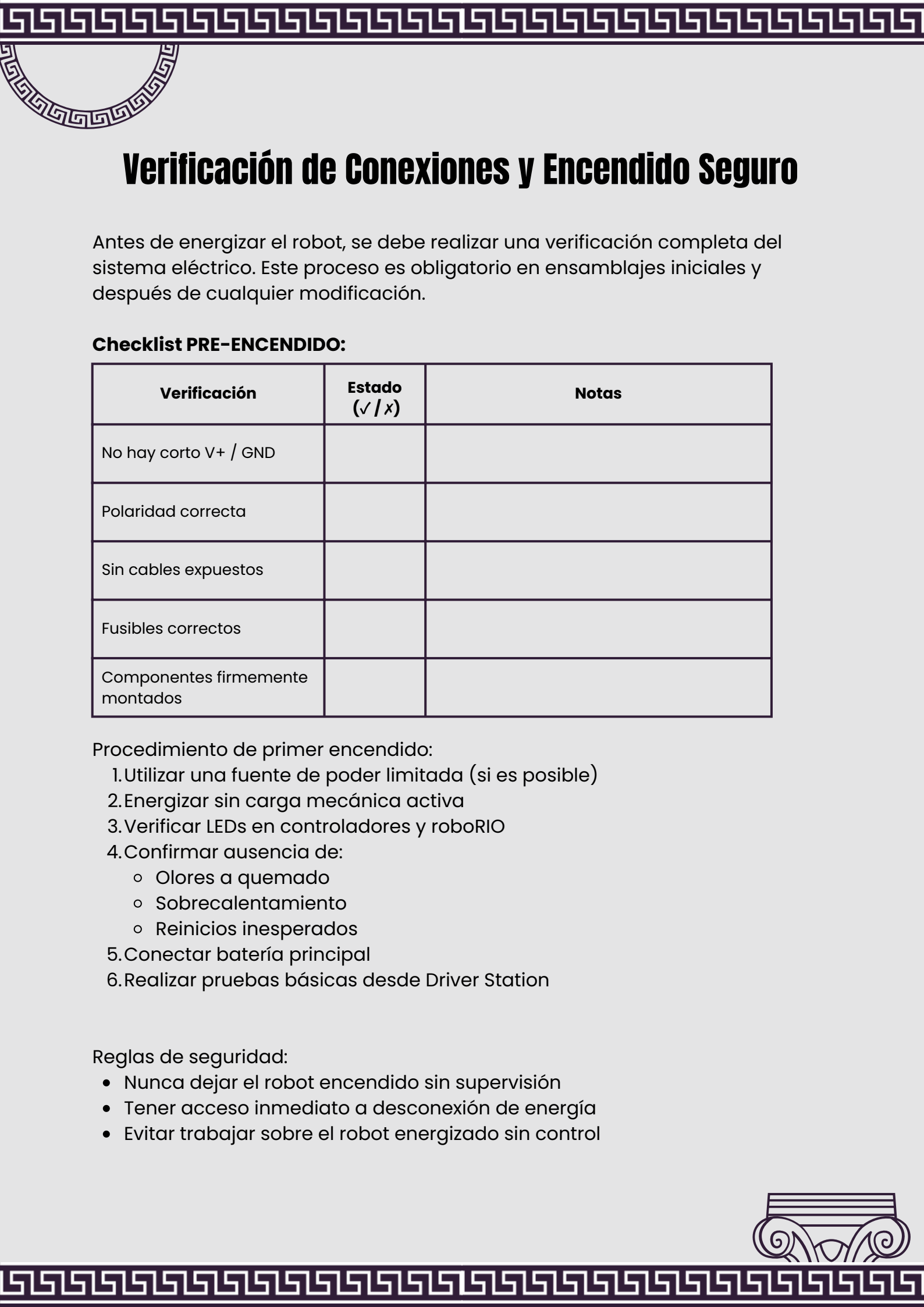
- **Ruteo:**
  - Alejar cables de partes móviles
  - Evitar bordes filosos
  - Separar potencia de señal cuando sea posible
- **Gestión:**
  - Uso de bridas o velcro
  - No forzar o jalar cables
  - Mantener orden por subsistemas
- **Conectores:**
  - Inserción completa
  - Uso de retención
  - Evitar conexiones flojas
- **Etiquetado (obligatorio):**
  - Etiquetar ambos extremos
  - Incluir nombre y función
  - Ejemplo: INTAKE\_MOTOR\_LEFT

## Calibres mínimos de cable (FRC):

| Aplicación          | Calibre mínimo (AWG) |
|---------------------|----------------------|
| Batería principal   | 6 AWG                |
| Motores principales | 12 AWG               |
| Señales y sensores  | 18–24 AWG            |

El uso de calibres incorrectos puede provocar sobrecalentamiento, caída de voltaje y fallas intermitentes.





# Verificación de Conexiones y Encendido Seguro

Antes de energizar el robot, se debe realizar una verificación completa del sistema eléctrico. Este proceso es obligatorio en ensamblajes iniciales y después de cualquier modificación.

## Checklist PRE-ENCENDIDO:

| Verificación                    | Estado<br>(✓ / ✗) | Notas |
|---------------------------------|-------------------|-------|
| No hay corto V+ / GND           |                   |       |
| Polaridad correcta              |                   |       |
| Sin cables expuestos            |                   |       |
| Fusibles correctos              |                   |       |
| Componentes firmemente montados |                   |       |

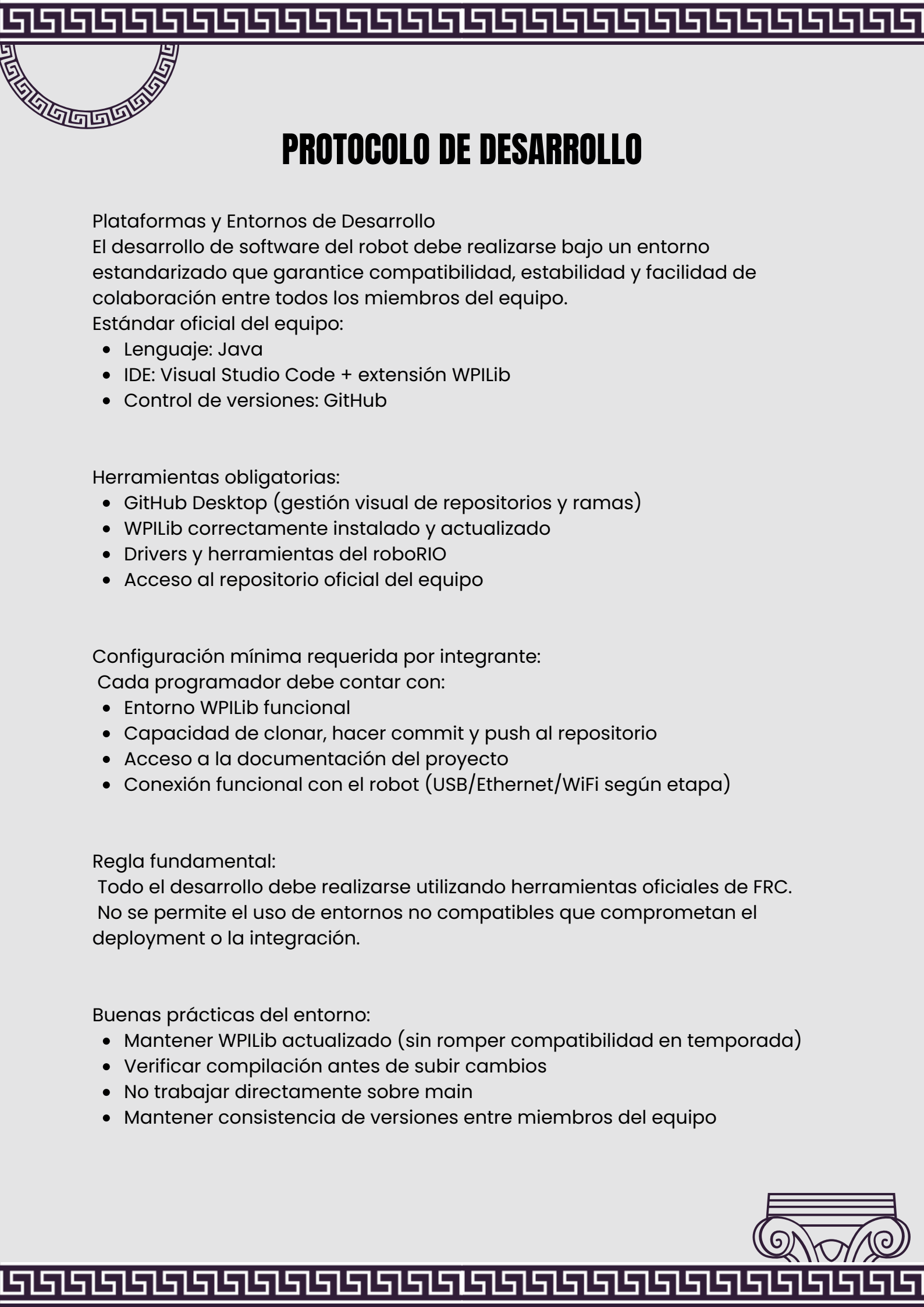
## Procedimiento de primer encendido:

- 1.Utilizar una fuente de poder limitada (si es posible)
- 2.Energizar sin carga mecánica activa
- 3.Verificar LEDs en controladores y roboRIO
- 4.Confirmar ausencia de:
  - Olores a quemado
  - Sobrecalentamiento
  - Reinicios inesperados
- 5.Conectar batería principal
- 6.Realizar pruebas básicas desde Driver Station

## Reglas de seguridad:

- Nunca dejar el robot encendido sin supervisión
- Tener acceso inmediato a desconexión de energía
- Evitar trabajar sobre el robot energizado sin control





# PROTOCOLO DE DESARROLLO

## Plataformas y Entornos de Desarrollo

El desarrollo de software del robot debe realizarse bajo un entorno estandarizado que garantice compatibilidad, estabilidad y facilidad de colaboración entre todos los miembros del equipo.

Estándar oficial del equipo:

- Lenguaje: Java
- IDE: Visual Studio Code + extensión WPILib
- Control de versiones: GitHub

Herramientas obligatorias:

- GitHub Desktop (gestión visual de repositorios y ramas)
- WPILib correctamente instalado y actualizado
- Drivers y herramientas del roboRIO
- Acceso al repositorio oficial del equipo

Configuración mínima requerida por integrante:

Cada programador debe contar con:

- Entorno WPILib funcional
- Capacidad de clonar, hacer commit y push al repositorio
- Acceso a la documentación del proyecto
- Conexión funcional con el robot (USB/Ethernet/WiFi según etapa)

Regla fundamental:

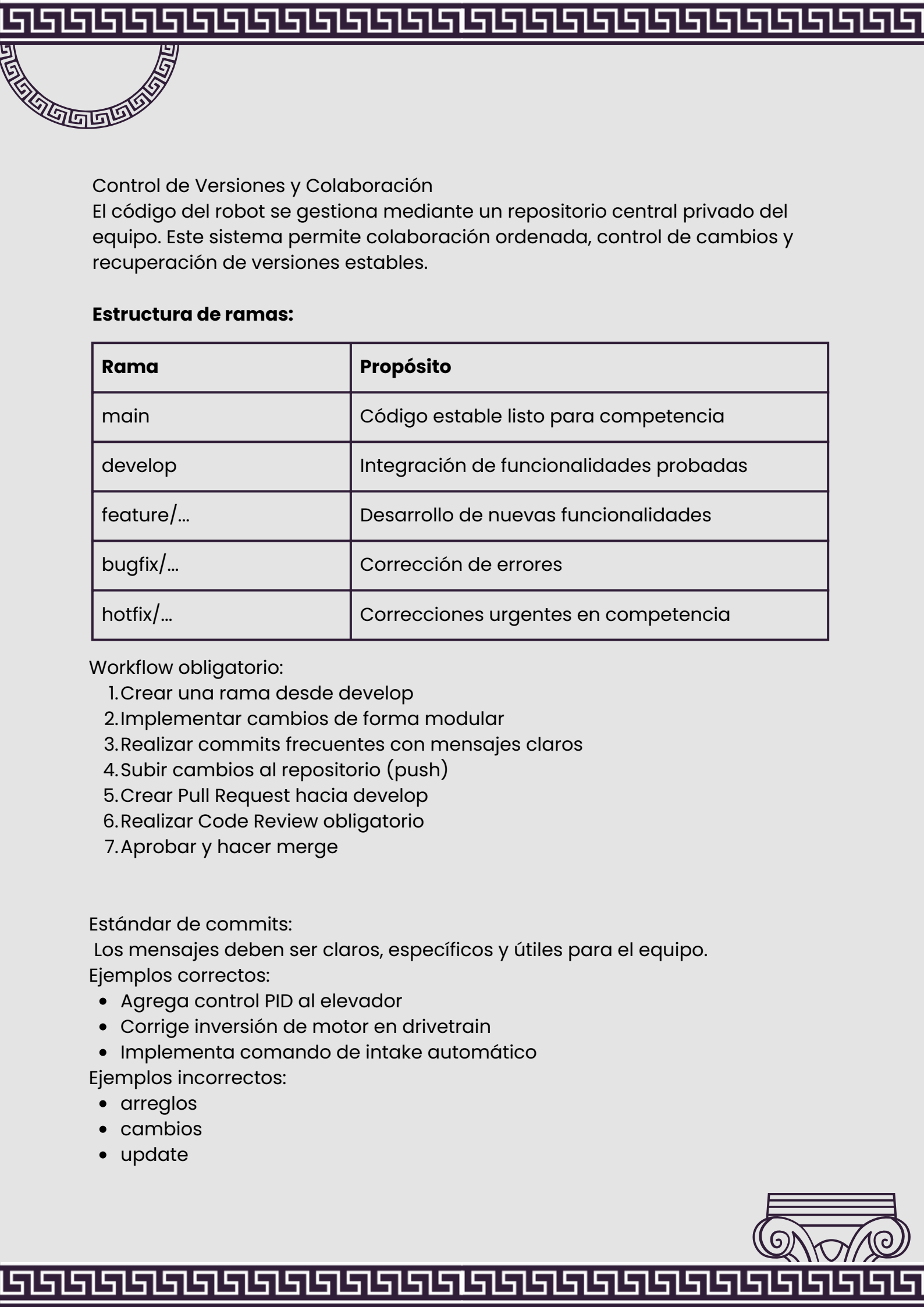
Todo el desarrollo debe realizarse utilizando herramientas oficiales de FRC.

No se permite el uso de entornos no compatibles que comprometan el deployment o la integración.

Buenas prácticas del entorno:

- Mantener WPILib actualizado (sin romper compatibilidad en temporada)
- Verificar compilación antes de subir cambios
- No trabajar directamente sobre main
- Mantener consistencia de versiones entre miembros del equipo





## Control de Versiones y Colaboración

El código del robot se gestiona mediante un repositorio central privado del equipo. Este sistema permite colaboración ordenada, control de cambios y recuperación de versiones estables.

### Estructura de ramas:

| Rama        | Propósito                               |
|-------------|-----------------------------------------|
| main        | Código estable listo para competencia   |
| develop     | Integración de funcionalidades probadas |
| feature/... | Desarrollo de nuevas funcionalidades    |
| bugfix/...  | Corrección de errores                   |
| hotfix/...  | Correcciones urgentes en competencia    |

### Workflow obligatorio:

1. Crear una rama desde develop
2. Implementar cambios de forma modular
3. Realizar commits frecuentes con mensajes claros
4. Subir cambios al repositorio (push)
5. Crear Pull Request hacia develop
6. Realizar Code Review obligatorio
7. Aprobar y hacer merge

### Estándar de commits:

Los mensajes deben ser claros, específicos y útiles para el equipo.

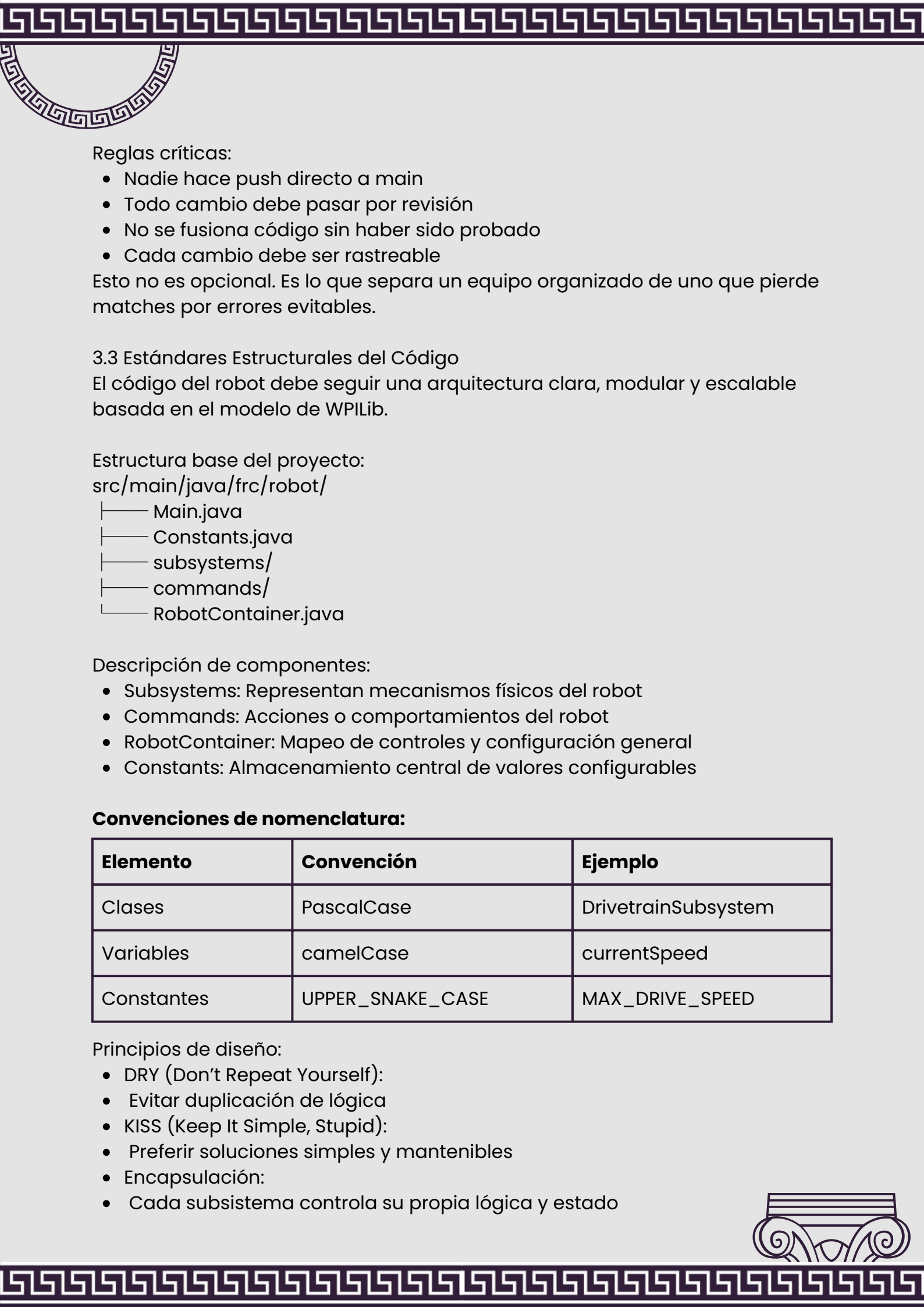
### Ejemplos correctos:

- Agrega control PID al elevador
- Corrige inversión de motor en drivetrain
- Implementa comando de intake automático

### Ejemplos incorrectos:

- arreglos
- cambios
- update





Reglas críticas:

- Nadie hace push directo a main
- Todo cambio debe pasar por revisión
- No se fusiona código sin haber sido probado
- Cada cambio debe ser rastreable

Esto no es opcional. Es lo que separa un equipo organizado de uno que pierde matches por errores evitables.

### 3.3 Estándares Estructurales del Código

El código del robot debe seguir una arquitectura clara, modular y escalable basada en el modelo de WPILib.

Estructura base del proyecto:

```
src/main/java/frc/robot/  
├── Main.java  
├── Constants.java  
├── subsystems/  
├── commands/  
└── RobotContainer.java
```

Descripción de componentes:

- Subsystems: Representan mecanismos físicos del robot
- Commands: Acciones o comportamientos del robot
- RobotContainer: Mapeo de controles y configuración general
- Constants: Almacenamiento central de valores configurables

#### Convenciones de nomenclatura:

| Elemento   | Convención       | Ejemplo             |
|------------|------------------|---------------------|
| Clases     | PascalCase       | DrivetrainSubsystem |
| Variables  | camelCase        | currentSpeed        |
| Constantes | UPPER_SNAKE_CASE | MAX_DRIVE_SPEED     |

Principios de diseño:

- DRY (Don't Repeat Yourself):
- Evitar duplicación de lógica
- KISS (Keep It Simple, Stupid):
- Preferir soluciones simples y mantenibles
- Encapsulación:
- Cada subsistema controla su propia lógica y estado





Buenas prácticas adicionales:

- No mezclar lógica de múltiples subsistemas en una sola clase
- Evitar valores "hardcodeados" fuera de Constants.java
- Mantener funciones cortas y claras
- Priorizar legibilidad sobre optimización prematura

### 3.4 Documentación Técnica y Engineering Notebook (CRÍTICO)

La documentación no es opcional. Es parte del proceso de ingeniería y evaluación del equipo.

Todo el desarrollo debe estar registrado en:

- Engineering Notebook
- Bitácora técnica del equipo

La documentación debe incluir:

- Decisiones de diseño (por qué se eligió una solución)
- Cambios importantes en el código
- Valores de tuning (PID, offsets, velocidades, etc.)
- Problemas encontrados
- Soluciones implementadas
- Resultados de pruebas

Documentación dentro del código:

El código debe ser entendible sin necesidad de explicación externa.

Reglas:

- Comentar el por qué, no el qué
- Documentar funciones importantes con Javadoc

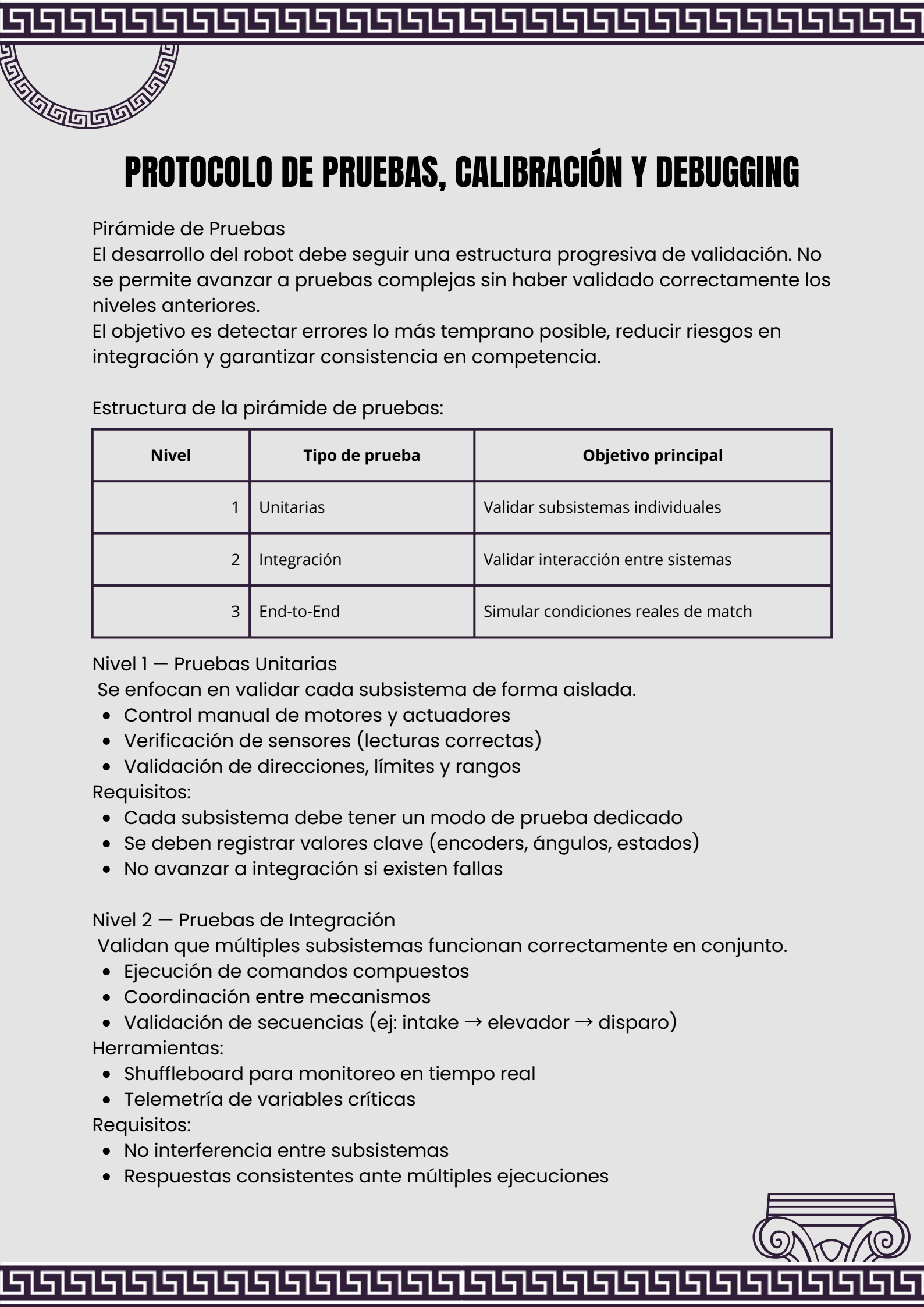
Ejemplo:

```
/**  
 * Calcula el ángulo óptimo de disparo basado en la distancia al objetivo.  
 * @param distance Distancia en metros medida por visión  
 * @return Ángulo en grados  
 */
```

Objetivo de la documentación:

- Permitir continuidad entre temporadas
- Facilitar debugging
- Acelerar capacitación de nuevos miembros
- Respalda decisiones técnicas ante jueces





# PROTOCOLO DE PRUEBAS, CALIBRACIÓN Y DEBUGGING

## Pirámide de Pruebas

El desarrollo del robot debe seguir una estructura progresiva de validación. No se permite avanzar a pruebas complejas sin haber validado correctamente los niveles anteriores.

El objetivo es detectar errores lo más temprano posible, reducir riesgos en integración y garantizar consistencia en competencia.

Estructura de la pirámide de pruebas:

| Nivel | Tipo de prueba | Objetivo principal                  |
|-------|----------------|-------------------------------------|
| 1     | Unitarias      | Validar subsistemas individuales    |
| 2     | Integración    | Validar interacción entre sistemas  |
| 3     | End-to-End     | Simular condiciones reales de match |

### Nivel 1 — Pruebas Unitarias

Se enfocan en validar cada subsistema de forma aislada.

- Control manual de motores y actuadores
- Verificación de sensores (lecturas correctas)
- Validación de direcciones, límites y rangos

Requisitos:

- Cada subsistema debe tener un modo de prueba dedicado
- Se deben registrar valores clave (encoders, ángulos, estados)
- No avanzar a integración si existen fallas

### Nivel 2 — Pruebas de Integración

Validan que múltiples subsistemas funcionan correctamente en conjunto.

- Ejecución de comandos compuestos
- Coordinación entre mecanismos
- Validación de secuencias (ej: intake → elevador → disparo)

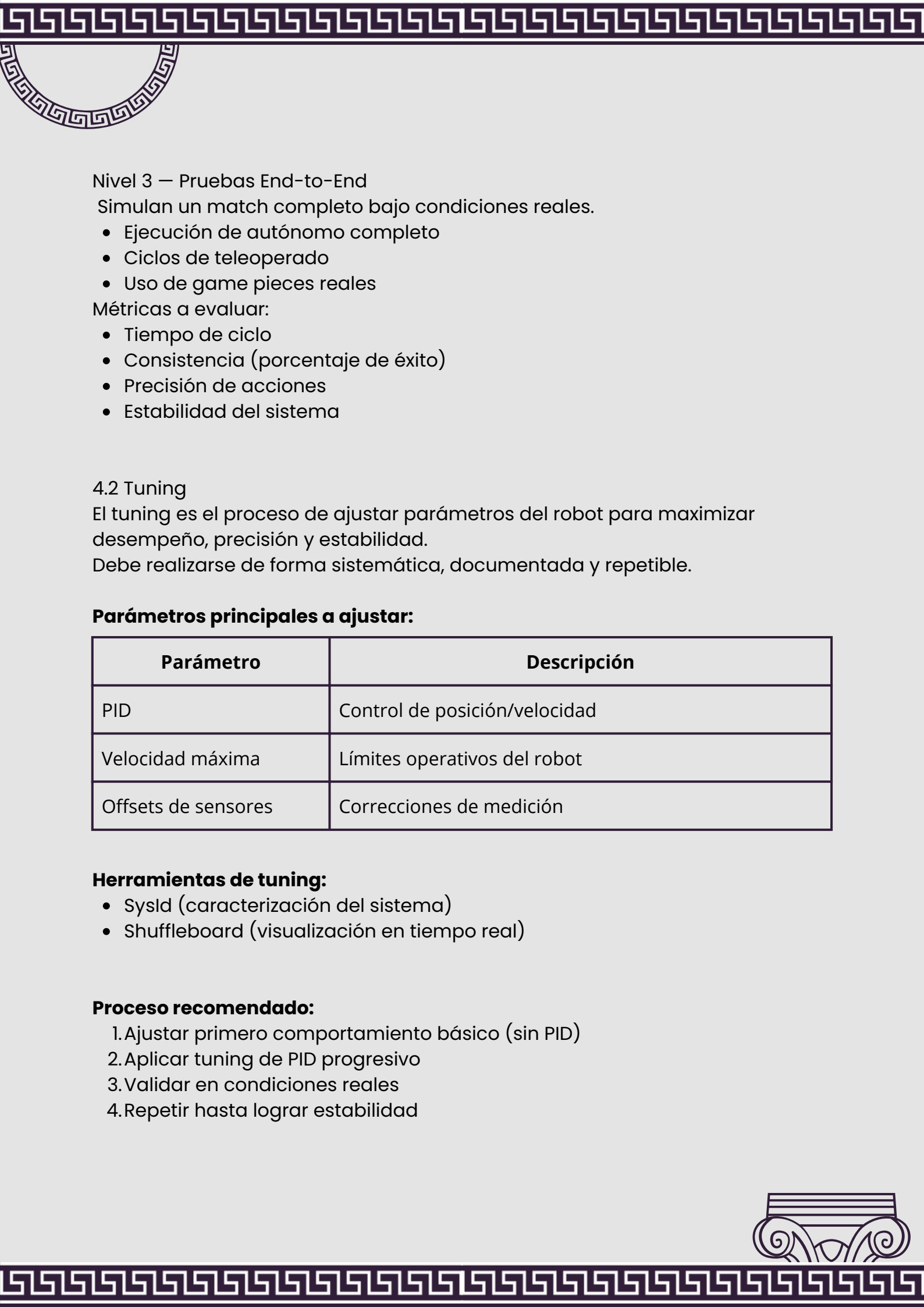
Herramientas:

- Shuffleboard para monitoreo en tiempo real
- Telemetría de variables críticas

Requisitos:

- No interferencia entre subsistemas
- Respuestas consistentes ante múltiples ejecuciones





Nivel 3 — Pruebas End-to-End

Simulan un match completo bajo condiciones reales.

- Ejecución de autónomo completo
- Ciclos de teleoperado
- Uso de game pieces reales

Métricas a evaluar:

- Tiempo de ciclo
- Consistencia (porcentaje de éxito)
- Precisión de acciones
- Estabilidad del sistema

4.2 Tuning

El tuning es el proceso de ajustar parámetros del robot para maximizar desempeño, precisión y estabilidad.

Debe realizarse de forma sistemática, documentada y repetible.

Parámetros principales a ajustar:

| Parámetro           | Descripción                   |
|---------------------|-------------------------------|
| PID                 | Control de posición/velocidad |
| Velocidad máxima    | Límites operativos del robot  |
| Offsets de sensores | Correcciones de medición      |

Herramientas de tuning:

- SysId (caracterización del sistema)
- Shuffleboard (visualización en tiempo real)

Proceso recomendado:

1. Ajustar primero comportamiento básico (sin PID)
2. Aplicar tuning de PID progresivo
3. Validar en condiciones reales
4. Repetir hasta lograr estabilidad





Reglas obligatorias:

- Todos los valores deben almacenarse en Constants.java
- Todos los cambios deben documentarse en bitácora
- No se permite tuning “al aire” sin registro

#### 4.3 Pruebas y Calibración de Autónomo

El desarrollo del autónomo debe priorizar confiabilidad sobre complejidad. Un autónomo simple que siempre funciona es mejor que uno avanzado que falla.

Estrategia de desarrollo:

- Construcción modular (comandos reutilizables)
- Uso de sensores (encoders, giroscopio) en lugar de timers
- Integración de visión para correcciones dinámicas

Buenas prácticas:

- Validar cada comando por separado antes de integrarlo
- Evitar dependencias innecesarias entre acciones
- Diseñar rutas simples y eficientes

Safe Auto (OBLIGATORIO):

Todo robot debe contar con un autónomo de respaldo que:

| Característica    | Descripción                     |
|-------------------|---------------------------------|
| Movimiento básico | Avanza una distancia controlada |
| Acción simple     | Ejecuta una tarea mínima útil   |
| Independencia     | Funciona sin sensores complejos |

Este autónomo debe poder ejecutarse en cualquier situación sin riesgo de fallo crítico.

Validación del autónomo:

- Pruebas repetidas (mínimo 10 ejecuciones consecutivas)
- Consistencia > 90%
- Evaluación en condiciones reales de campo





#### 4.4 Fine Tuning

El fine tuning consiste en realizar ajustes finales bajo condiciones reales de operación para optimizar el rendimiento del robot.

Objetivos:

- Reducir tiempos de ciclo
- Mejorar precisión
- Aumentar consistencia

Aspectos a optimizar:

- Velocidades de mecanismos
- Transiciones entre acciones
- Tiempos de respuesta
- Ajustes finos de PID

Metodología:

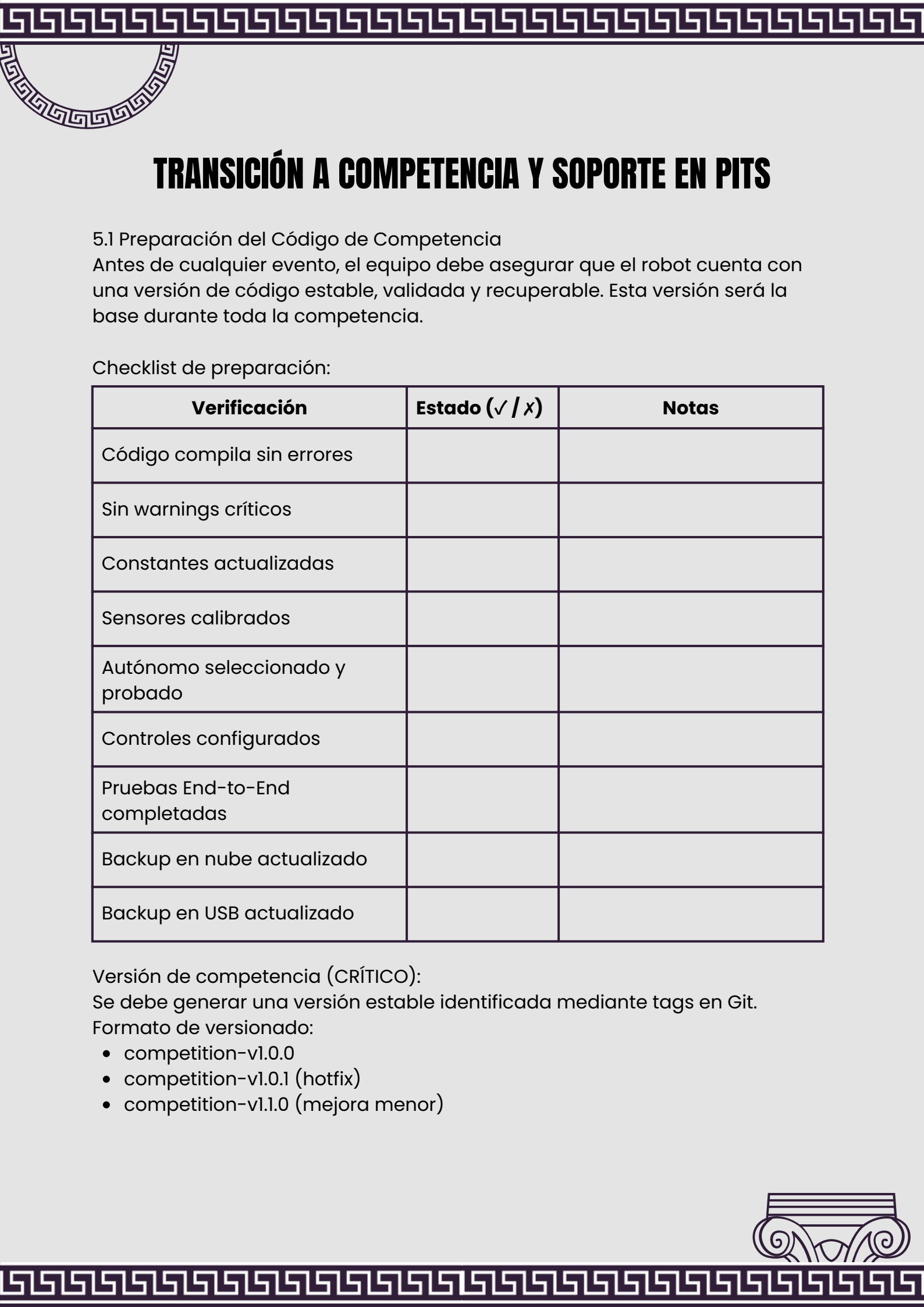
- Realizar cambios pequeños y controlados
- Probar inmediatamente después de cada ajuste
- Comparar resultados con métricas previas

Regla clave:

La consistencia es más importante que la velocidad máxima.

Un robot ligeramente más lento pero confiable gana más matches que uno rápido pero inestable.





# TRANSICIÓN A COMPETENCIA Y SOPORTE EN PITS

## 5.1 Preparación del Código de Competencia

Antes de cualquier evento, el equipo debe asegurar que el robot cuenta con una versión de código estable, validada y recuperable. Esta versión será la base durante toda la competencia.

Checklist de preparación:

| Verificación                    | Estado (✓ / x) | Notas |
|---------------------------------|----------------|-------|
| Código compila sin errores      |                |       |
| Sin warnings críticos           |                |       |
| Constantes actualizadas         |                |       |
| Sensores calibrados             |                |       |
| Autónomo seleccionado y probado |                |       |
| Controles configurados          |                |       |
| Pruebas End-to-End completadas  |                |       |
| Backup en nube actualizado      |                |       |
| Backup en USB actualizado       |                |       |

Versión de competencia (CRÍTICO):

Se debe generar una versión estable identificada mediante tags en Git.

Formato de versionado:

- competition-v1.0.0
- competition-v1.0.1 (hotfix)
- competition-v1.1.0 (mejora menor)





Esto permite:

- Volver a una versión funcional en segundos
- Evitar perder código estable
- Mantener control durante cambios en competencia

Reglas antes de competencia:

- No se hacen cambios de última hora sin pruebas completas
- Todo el código en main debe ser estable
- Se debe poder desplegar el robot desde cero en cualquier momento

Preparación del entorno:

- Laptop principal configurada y probada
- Proyecto compila sin conexión a internet
- Herramientas necesarias instaladas y funcionales

## 5.2 Checklist para Pits y Soporte en Competencia

Durante la competencia, el objetivo es resolver problemas rápido, sin errores y sin improvisación.

Equipo mínimo en pits:

| Elemento                  | Estado (✓ / x) | Notas |
|---------------------------|----------------|-------|
| Laptop de desarrollo      |                |       |
| Cables (USB / Ethernet)   |                |       |
| USB con respaldo completo |                |       |
| Instaladores WPILib       |                |       |
| Drivers del roboRIO       |                |       |
| Código de competencia     |                |       |
| Notas de troubleshooting  |                |       |





Contenido obligatorio del USB de respaldo:

- Código fuente de la versión de competencia
- Instaladores de WPILib
- Drivers necesarios
- Versiones anteriores funcionales
- Documentación básica del sistema

Protocolo de cambios en competencia (CRÍTICO)

Durante un evento, cualquier cambio debe ser controlado. Cambios sin proceso son la causa más común de fallas en un match.

Proceso obligatorio:

1. Detectar problema y definir causa probable
2. Validar necesidad real del cambio
3. Aprobar cambio con el Líder de Software
4. Crear rama hotfix/... desde versión actual
5. Implementar cambio mínimo necesario
6. Probar inmediatamente en robot
7. Validar funcionamiento completo (no solo el fix)
8. Hacer merge y generar nuevo tag
9. Actualizar respaldo en USB

Reglas críticas:

- No hacer cambios grandes en competencia
- No probar código directamente en match
- Siempre probar antes de subir
- Todo cambio debe ser reversible

Diagnóstico rápido en pits

El equipo debe ser capaz de identificar fallas en minutos.

Problemas comunes a verificar:

- Conexión con robot (Driver Station)
- Estado del roboRIO
- CAN bus activo
- Sensores reportando datos
- Código desplegado correctamente





# DOCUMENTACIÓN

## 6.1 Propósito de la Documentación

La documentación del área de programación y electrónica tiene como objetivo:

- Registrar decisiones técnicas y su justificación
- Garantizar continuidad entre temporadas
- Facilitar debugging y mantenimiento
- Preparar evidencia sólida para jueces
- Estandarizar el conocimiento dentro del equipo

Toda la documentación debe ser clara, accesible, actualizada y útil, no solo decorativa.

## 6.2 Engineering Notebook (Libro de Ingeniería)

El Engineering Notebook es el registro oficial del proceso de ingeniería del equipo y debe incluir de forma clara todo lo relacionado con software y electrónica.

Contenido obligatorio para programación y electrónica:

- Arquitectura del código (estructura del proyecto)
- Diagrama del sistema eléctrico
- Integración hardware-software
- Decisiones de diseño (ej: uso de sensores, lógica de control)
- Estrategia de autónomo
- Descripción de subsistemas y su control

Registro de desarrollo:

- Iteraciones de código
- Problemas encontrados
- Soluciones implementadas
- Cambios importantes en el sistema

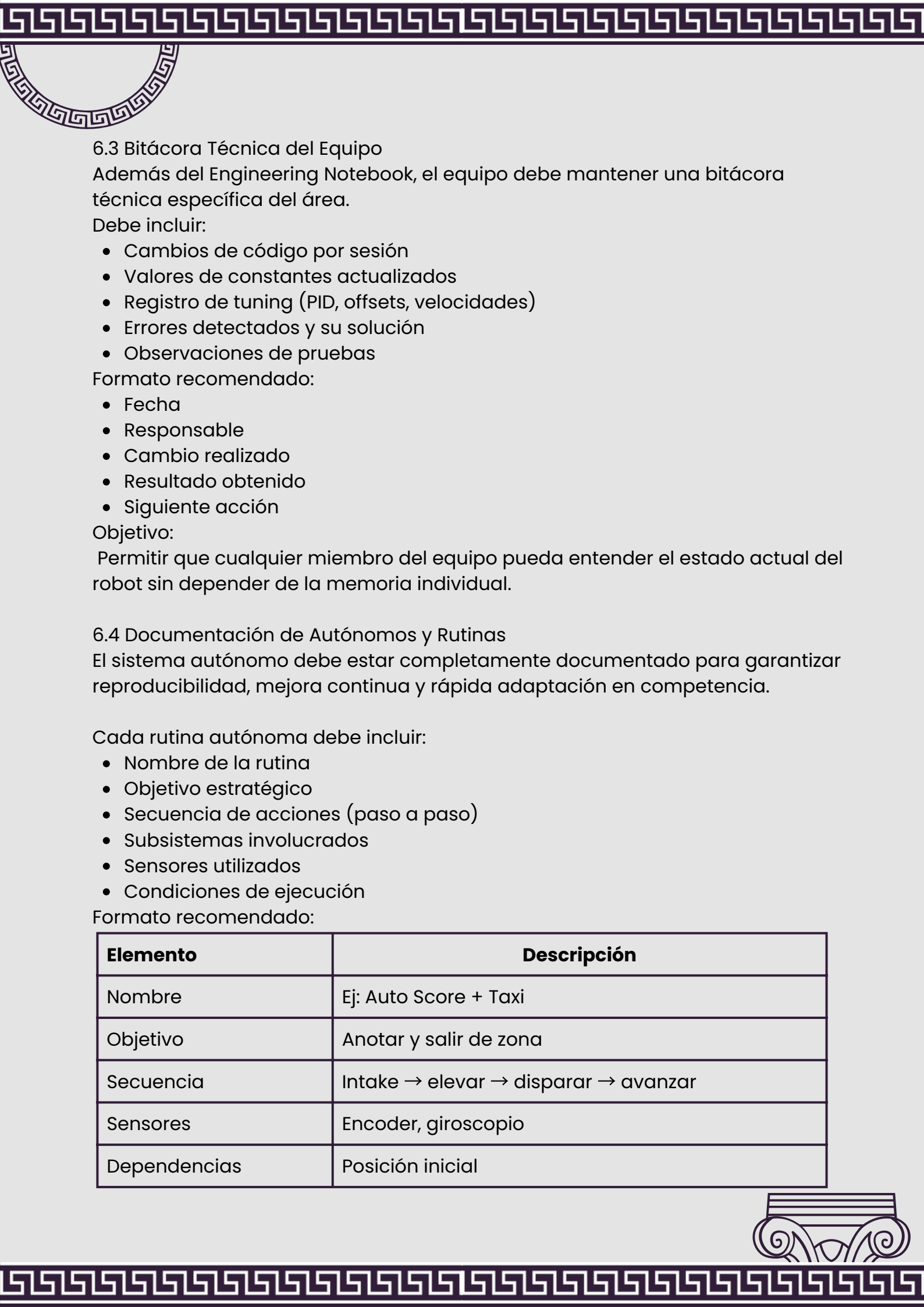
Registro de pruebas:

- Resultados de pruebas unitarias, integración y End-to-End
- Valores obtenidos en tuning
- Ajustes realizados y resultados

Buenas prácticas:

- Incluir diagramas, imágenes y capturas
- Mantener orden cronológico
- Explicar el “por qué” de las decisiones





### 6.3 Bitácora Técnica del Equipo

Además del Engineering Notebook, el equipo debe mantener una bitácora técnica específica del área.

Debe incluir:

- Cambios de código por sesión
- Valores de constantes actualizados
- Registro de tuning (PID, offsets, velocidades)
- Errores detectados y su solución
- Observaciones de pruebas

Formato recomendado:

- Fecha
- Responsable
- Cambio realizado
- Resultado obtenido
- Siguiente acción

Objetivo:

Permitir que cualquier miembro del equipo pueda entender el estado actual del robot sin depender de la memoria individual.

### 6.4 Documentación de Autónomos y Rutinas

El sistema autónomo debe estar completamente documentado para garantizar reproducibilidad, mejora continua y rápida adaptación en competencia.

Cada rutina autónoma debe incluir:

- Nombre de la rutina
- Objetivo estratégico
- Secuencia de acciones (paso a paso)
- Subsistemas involucrados
- Sensores utilizados
- Condiciones de ejecución

Formato recomendado:

| Elemento     | Descripción                          |
|--------------|--------------------------------------|
| Nombre       | Ej: Auto Score + Taxi                |
| Objetivo     | Anotar y salir de zona               |
| Secuencia    | Intake → elevar → disparar → avanzar |
| Sensores     | Encoder, giroscopio                  |
| Dependencias | Posición inicial                     |





Adicional:

- Diagrama de trayectoria (si aplica)
- Código asociado o referencia de comandos
- Resultados de pruebas y consistencia

## 6.5 Documentación de Código y Sistema

El código debe ser autoexplicativo y complementado con documentación técnica.

Requisitos:

- Uso de comentarios explicando el “por qué”
- Uso de Javadoc en funciones clave
- Organización clara por subsistemas

Se debe documentar:

- Función de cada subsistema
- Relación entre comandos
- Uso de sensores
- Lógica de control

Constants.java (CRÍTICO):

- Todas las constantes deben estar centralizadas
- Cada valor debe tener contexto documentado
- Cambios deben registrarse en bitácora

## 6.6 Tech Binder (Opcional)

El Tech Binder es un respaldo físico o digital diseñado para uso en pits y presentaciones.

Debe incluir:

- Diagrama eléctrico
- Arquitectura del código
- Lista de subsistemas
- Rutinas autónomas
- Troubleshooting básico
- Versiones de código relevantes

Objetivo:

- Acceso rápido a información crítica en competencia
- Soporte para debugging
- Apoyo en entrevistas con jueces





## 6.7 Respaldo y Control de Información

Toda la documentación debe estar respaldada y accesible en todo momento.

Requisitos de respaldo:

- Copia en la nube (Drive, Notion, etc.)
- Copia local (USB de competencia)
- Acceso para miembros clave del equipo

Contenido mínimo del respaldo:

- Código fuente
- Documentación del proyecto
- Engineering Notebook (digital o escaneado)
- Rutinas autónomas
- Instaladores y herramientas

Reglas:

- Actualizar respaldos antes de cada competencia
- Verificar acceso sin internet
- Mantener versiones organizadas

